SoftwareX 31 (2025) 102314

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

Original software publication

WireGuard-AES: Hardware based encryption to WireGuard for VPN gateways

M.F. Yuce [a], O. Keskin [a], E. Yerlikaya [a], M.Y. Akmaz [a], A. Kirca [a], D. Yiltas-Kaplan [b], M.A. Erturk [c,d], Z. Gurkas-Aydin [b], Ö.C. Turna [b,*], M.T. Buyukakkaslar [d], Ş. Durukan-Odabaşı [b], M.A. Aydin [b]

[a] Ulak Haberlesme A.S, Ankara, Turkiye
[b] Istanbul University-Cerrahpasa, Department of Computer Engineering, Istanbul, Turkiye
[c] Edinburgh Napier University, School of Computing, Engineering and The Built Environment, Edinburgh, UK
[d] Istanbul University, Department of Computer Engineering, Istanbul, Turkiye

ARTICLE INFO

Keywords:
AES
Throughput
Virtual private network
VPN
WireGuard

ABSTRACT

WireGuard is a high-performance virtual private network (VPN) implemented in the Linux kernel, known for its speed and software-based encryption. However, it struggles as a VPN gateway (VPNGW) due to reduced throughput when multiple clients connect—especially in software-defined networks (SDNs), where hardware encryption support is underutilized. This study introduces a novel WireGuard implementation using Advanced Encryption Standard (AES) encryption, leveraging hardware support to improve performance. Kernel-based AES boosts throughput by 11%, reduces retransmissions by 5.5%, and lowers central processing unit (CPU) usage by at least 2% (with 95% confidence interval). User-space AES achieves up to 19% higher throughput on modern CPUs, paving the way for increased speeds and better efficiency with larger maximum transmission units (MTUs).

Code metadata

Current code version	wireguard-5.10.55
Permanent link to code/repository used for this code version	https://github.com/ElsevierSoftwareX/SOFTX-D-25-00258[a] [b]
Permanent link to Reproducible Capsule	https://codeocean.com/capsule/0270963/tree/v1
Legal Code License	GNU General Public License v2.0
Code versioning system used	git
Software code languages, tools, and services used	C, bash
Compilation requirements, operating environments & dependencies	
If available Link to developer documentation/manual	
Support email for questions	fatih.yuce@ulakhaberlesme.com.tr

[a] Boringtun implementation is available at: https://github.com/UlakCommunications/boringtun-aes.

[b] Validation data is available at: "experimentation/experiment_archieve" folder.

Software metadata

Current software version	5.10.55
Permanent link to executables of this version	For Linux 5.4: https://github.com/UlakCommunications/wireguard-aes/releases
Permanent link to Reproducible Capsule	
Legal Software License	GNU General Public License v2.0
Computing platforms/Operating Systems	WireGuard running in the Linux kernel with Advanced Encryption Standard (AES) support
Installation requirements & dependencies	
If available, link to user manual—if formally published include a reference to the publication in the reference list	
Support email for questions	fatih.yuce@ulakhaberlesme.com.tr

* Corresponding author.
E-mail addresses: fatih.yuce@ulakhaberlesme.com.tr (M.F. Yuce), ozgur.keskin@ulakhaberlesme.com.tr (O. Keskin), enes.yerlikaya@ulakhaberlesme.com.tr (E. Yerlikaya), mustafa.akmaz@ulakhaberlesme.com.tr (M.Y. Akmaz), ali.kirca@ulakhaberlesme.com.tr (A. Kirca), dyiltas@iuc.edu.tr (D. Yiltas-Kaplan), M.Erturk@napier.ac.uk, mehmetali.erturk@istanbul.edu.tr (M.A. Erturk), zeynepg@iuc.edu.tr (Z. Gurkas-Aydin), ozgurcan.turna@iuc.edu.tr (Ö.C. Turna), mbuyukakkaslar@ogr.iu.edu.tr (M.T. Buyukakkaslar), safak.odabasi@iuc.edu.tr (Ş. Durukan-Odabaşı), aydinali@iuc.edu.tr (M.A. Aydin).

https://doi.org/10.1016/j.softx.2025.102314
Received 21 April 2025; Received in revised form 11 August 2025; Accepted 12 August 2025
Available online 21 August 2025
2352-7110/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Motivation and significance

WireGuard is an Open Systems Interconnection (OSI) layer 3, open-source virtual private network (VPN) protocol built into modern kernels, using the User Datagram Protocol (UDP) to encapsulate Internet Protocol (IP) traffic [1]. While UDP offers high speed, it may lead to Transmission Control Protocol (TCP) packet loss during retransmissions. WireGuard is praised for its simplicity, cryptographic soundness, and performance. It employs minimal primitives (ChaCha20, Curve25519, Noise), to ensure secure communication [2,3]. Its small codebase (<1000 lines) enhances auditability and security [4,5].

WireGuard's software-based encryption offers broad platform compatibility but imposes performance limitations on gateways serving numerous clients. While most Advanced RISC Machines (ARM) devices lack hardware support for the Advanced Encryption Standard (AES), many software-defined network (SDN) gateways (SDNGWs) and VPN gateways (VPNGWs) incorporate Intel Advanced Encryption Standard New Instructions (AES-NI) for hardware acceleration. Nevertheless, performance on the server side (i.e., the VPNGW) deteriorates as client numbers increase (a challenge not mitigated by containerization due to shared kernels). Deploying multiple virtual machines (VMs) remains a costly alternative.

This study integrates AES-based encryption into WireGuard to leverage hardware support, thereby enhancing throughput and reducing retransmissions. The approach is especially beneficial for VPN gateways (VPNGWs) in software-defined network (SDN) environments. Its two main contributions are: (1) an AES-accelerated WireGuard implementation and (2) benchmarks demonstrating superior kernel- and user-space performance.

This study presents a novel approach not previously addressed in the literature. Prior hardware-based efforts, typically using field-programmable gate arrays (FPGAs), involve costly, specialized hardware and limited Linux compatibility. In contrast, our method modifies a minimal subset of the Linux kernel, maintaining WireGuard compatibility and simplifying deployment.

To the best of our knowledge, no WireGuard implementation employs dedicated hardware-only encryption beyond FPGA-based designs. A comparative analysis with Internet Protocol Security (IPSec), which also benefits from hardware acceleration, appears in [6,7].

Initially a standalone kernel module, WireGuard was merged into the mainline Linux kernel in version 5.6, improving performance via reduced context switching and enhanced CPU cache locality.

The Extended Berkeley Packet Filter (eBPF) allows safe, verifiable code to be loaded into the kernel, while the eXpress Data Path (XDP) enables fast packet processing at the network interface card (NIC) level (even before packets reach the kernel networking stack). These features significantly boost performance for VPNs such as WireGuard, particularly when using `XDP_REDIRECT`[1] or `XDP_TX`.[2] Flame graphs in Fig. A.1 illustrate that kernel networking overhead dominates processing time (only 7% is spent on encryption and decryption).

Hardware-accelerated VPNs have been studied in both IPsec and WireGuard contexts. *Fast IPsec* achieves high throughput via kernel-level optimization and cryptographic hardware [8]. Other studies demonstrate FPGA-based acceleration and IPsec VPNs tailored for home gateways [9,10].

High-performance frameworks such as hXDP [11] and FD.io's VPP-integrated strongSwan [12] offer alternative paths to terabit-level secure tunnel (TbLST) performance. While WireGuard cannot currently achieve TbLST within a single VPN gateway (VPNGW), hardware-based implementations (such as the one presented in this study) are attracting growing interest.

In bandwidth-constrained settings such as EDGE, WireGuard remains effective due to its simplicity and maintainability. For stable core networks (CN), IPsec remains preferred for TbLST. A comprehensive survey appears in Shantharama et al. [13].

Disabling connection tracking (conntrack in Linux) further improves throughput (by up to 30% in our tests) as it is unnecessary prior to decryption in UDP-tunneled traffic. Cilium's eBPF/XDP-based conntrack system[4] shows potential for further optimizations.[3,4]

User-space WireGuard implementations must meet basic protocol requirements, such as support for command-line interface (CLI) usage and user-space application programming interface (UAPI) commands. Notable implementations include WireGuard-go,[5] WireGuard-rs,[6] and BoringTun.[7]

Although described as "*iffy*",[8] `WireGuard-go` remains the most stable and widely used user-space implementation, maintained by Jason A. Donenfeld. In our tests, a Tailscale fork achieved the highest throughput in single-process mode by utilizing all available CPU cores. While Tailscale's benchmark reported up to 42.4 Gb/s throughput with NIC offloading,[9] our experiments achieved 56 Gb/s on high-end CPUs. This version adds support for TCP Segmentation Offload (TSO), Generic Receive Offload (GRO), and the `sendmmsg/recvmmsg` system calls; however, these modifications have not been upstreamed. The default version lacks options for CPU or thread affinity control and, by default, utilizes all available processor cores.[10]

`WireGuard-rs`, a Rust-based implementation, appears abandoned (likely due to performance limitations inherent to certain Rust structures). While `BoringTun` performs poorly in its original form, a lightly modified fork achieves the best single-core performance. User-space frameworks like the Vector Packet Processor (VPP) and Data Plane Development Kit (DPDK) further enhance scalability by bypassing kernel-level bottlenecks.

Several bottlenecks in WireGuard (such as the kernel networking stack, *conntrack*, firewalls, CPU cache locality, and encryption) have been identified in prior works [14–16]. Kernel-based VPNs are limited in scalability and cannot efficiently support thousands of 1Gb/s clients on a single Linux host. As discussed in Section 3, user-space implementations are preferred since they bypass kernel-layer constraints.

Cache locality presents further challenges in user-space WireGuard, particularly on multi-core systems. While kernel-mode implementations manage cache locality more effectively, user-space versions benefit from core pinning. Notably, encryption is not the dominant bottleneck (accounting for only 7% of processing time [14,15]). Fig. 1 show how throughput initially declines with client load, stabilizing and recovering after client disconnections near the 1000-second mark.

1.1. Kernel packet path and cache considerations

The WireGuard kernel module operates entirely within the Linux networking stack, interfacing directly with the Netfilter and routing subsystems. Our integration of AES-256 in Galois/Counter Mode (GCM) hooks into the data transmission and reception paths via the functions `encrypt_packet()` and `decrypt_packet()`, as detailed in Section 2.2. These paths are invoked after routing decisions have

[1] `XDP_REDIRECT` is a return code used in XDP programs to redirect a packet to another network interface or socket, bypassing the normal Linux networking stack.

[2] `XDP_TX` is an XDP return code used to transmit a packet back out the same network interface it arrived on, bypassing the kernel networking stack.

[3] https://github.com/cilium/cilium/blob/main/bpf/lib/conntrack.h

[4] https://cilium.io/blog/2021/05/11/cni-benchmark/

[5] https://git.zx2c4.com/wireguard-go/

[6] https://git.zx2c4.com/wireguard-rs/

[7] https://github.com/cloudflare/boringtun

[8] https://lore.kernel.org/wireguard/CAHmME9qsK5Mt9nwHVOUf7i043TDBpHER4rt=Z9AAHjNhxVLeHQ@mail.gmail.com/

[9] https://tailscale.com/blog/throughput-improvements

[10] https://github.com/WireGuard/wireguard-go

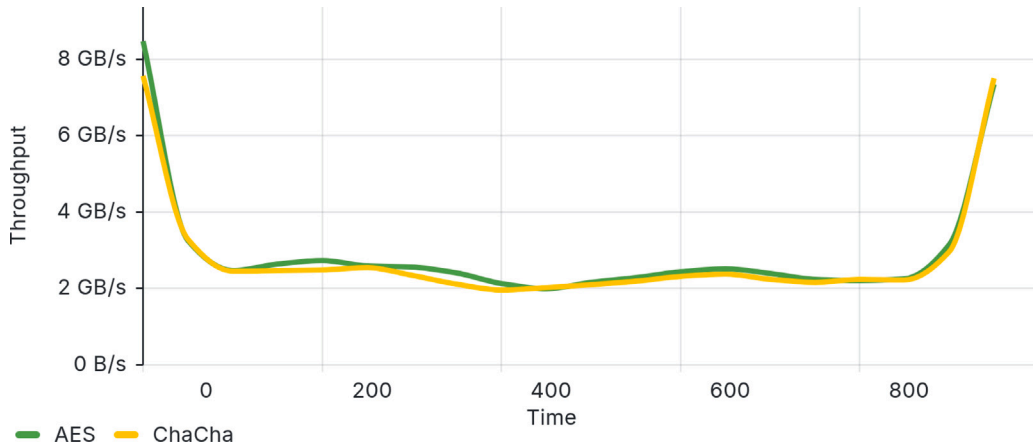

Fig. 1. Throughput on the sender side using kernel-mode WireGuard with AES and ChaCha20 encryption.

been made but before packets are passed to the Netfilter or connection tracking (conntrack) subsystems, depending on the system's configuration.

We note that WireGuard does not rely on `conntrack` by default. This behavior is advantageous in high-throughput VPN contexts, as it avoids the overhead of per-flow state maintenance. However, if `conntrack` is enabled on interfaces (e.g., via `iptables` or `nftables`), packets may still traverse the Netfilter hooks, potentially impacting throughput.

From a caching perspective, kernel-space cryptographic operations impose non-trivial instruction and data cache pressure. When encryption, routing, and network interface card (NIC) transmission tasks are distributed across different CPU cores, overall performance may degrade due to increased cache line bouncing and non-uniform memory access (NUMA) effects. In our evaluations, we observed a decline in throughput during multi-threaded scenarios in which interrupt handling and cryptographic computations contended for the shared last-level cache (LLC, typically L3). Consequently, strategies such as interrupt request (IRQ) affinity tuning and CPU pinning can have a significant impact on maintaining performance consistency in real-world deployments.

In our experiments, we did not apply any manual `conntrack` bypassing. Instead, we relied on the default WireGuard behavior and explicitly disabled Generic Receive Offload (GRO) and Large Receive Offload (LRO) to more accurately expose raw packet path latencies.

1.2. WireGuard implementations and AES support

WireGuard has multiple user-space implementations, each with distinct characteristics (Table 1). In this study, we compare three common variants: `WireGuard-go`, `BoringTun`, and our hardware-accelerated implementation.

`WireGuard-go` is the original Go-based version, widely used for cross-platform support. `BoringTun` is a Rust-based implementation developed by Cloudflare, with an emphasis on portability and memory safety. Both utilize ChaCha20-Poly1305 as the default cipher suite.

In contrast, our implementation leverages the kernel-based WireGuard module with AES-GCM encryption, accelerated through AES-NI hardware support when available. This configuration provides significantly improved performance on x86 platforms equipped with modern CPUs (see Table 1).

Our evaluation demonstrates the performance advantages of using AES hardware support, particularly in high-throughput VPN gateway scenarios where CPU efficiency and throughput are critical.

1.3. Fallback behavior

While our solution is optimized for systems with AES-NI, compatibility with systems lacking hardware AES support is preserved via a runtime fallback mechanism. If the kernel module fails to initialize AES-GCM (due to missing hardware acceleration or an unavailable cryptographic backend) it transparently falls back to ChaCha20-Poly1305 using the kernel's `chacha20-poly1305` implementation.

Although our implementation integrates AES-GCM into the WireGuard kernel module, it currently replaces the default ChaCha20-Poly1305 cipher and does not support cipher negotiation. As a result, both communicating peers must use the same cipher (either AES-GCM or ChaCha20-Poly1305) for the handshake and subsequent data exchange to succeed. Standard WireGuard clients, which exclusively use ChaCha20-Poly1305, are not interoperable with our AES-GCM implementation unless modified accordingly.

Achieving full interoperability across heterogeneous endpoints would require a protocol extension to support dynamic cipher negotiation within the WireGuard handshake (an avenue we identify as promising future work). In our VPNGW deployment scenario, this constraint poses no practical limitation, as all participating hardware platforms support AES-NI.

1.4. Performance profiling and flame graphs

To better understand system behavior under load, we used flame graphs to visualize CPU cycle distribution during encrypted packet processing.

Fig. A.1 illustrate CPU activity with AES hardware acceleration enabled. The tallest frames represent functions that consume the most CPU cycles. Functions associated with kernel cryptographic modules and Netfilter are prominently visible.

For less technical readers, flame graphs can be interpreted as hierarchical stacks: the width of each frame corresponds to the time spent in that function, while the Y-axis indicates stack depth (i.e., call hierarchy). This form of visualization is useful for identifying performance bottlenecks and opportunities for optimization.

1.5. Baseline comparison: IPsec and AES-GCM

To contextualize the benefits of AES-GCM acceleration in WireGuard, we reference a comprehensive study [17], which evaluated `WireGuard-c` (kernel), `WireGuard-go`, OpenVPN, and strongSwan with AES-GCM on 1 Gbps links. Their findings include:

- strongSwan with AES-GCM achieved up to 906 Mbps goodput and approximately 42% CPU usage;

Table 1
Comparison of WireGuard Implementations.

Implementation	Language	AES hardware support	Platform
`WireGuard-go`	Go	No	Cross-platform
`BoringTun`	Rust	Partial (software fallback)	Cross-platform
HW-Accelerated WireGuard	C (Kernel)	Yes (AES-NI)	Linux (x86 preferred)

Table 2
Illustrative VPN performance baseline.
Source: Adapted from [17].

Implementation	Goodput (Mbps)	CPU usage (%)
strongSwan (AES-GCM)	906	~42%
WireGuard-C	901	Higher than strongSwan
WireGuard-Go	~916	~268%

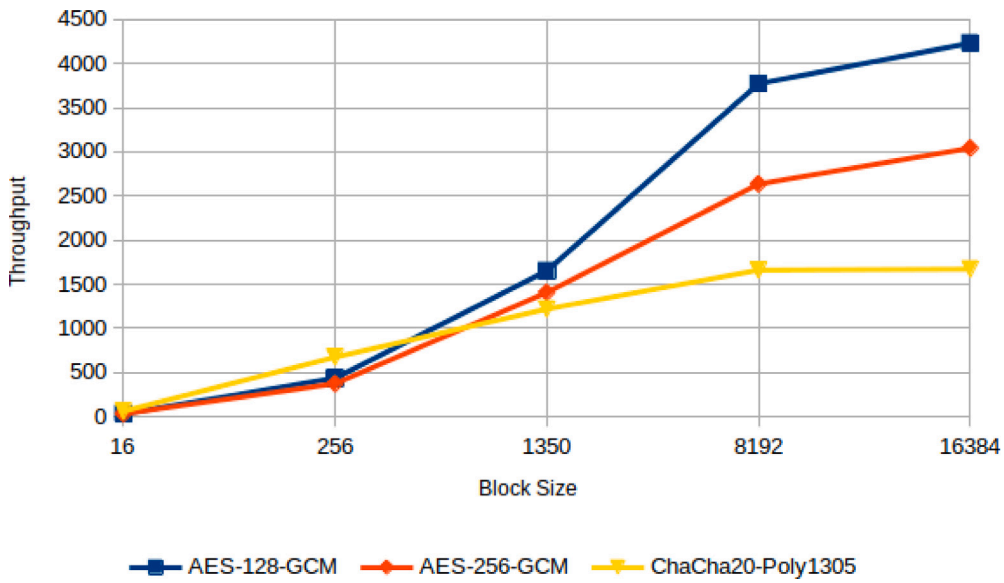

Fig. 2. Encryption throughput (MB/s, with AES-NI) versus block size (bytes).

- `WireGuard-c` closely followed at 901 Mbps with moderately higher CPU utilization;
- `WireGuard-go` exhibited significantly higher CPU usage (approximately 5–6× greater than strongSwan).

As shown in Table 2, these results indicate that IPsec with AES-GCM can achieve throughput comparable to WireGuard on AES-NI-capable hardware, albeit with differing CPU utilization profiles [17] (see Table 2).

2. Software description

Before hardware support for WireGuard was integrated into the Linux kernel, the performance of various encryption algorithms was evaluated. As shown in Fig. 2, AES with AES-NI significantly outperforms alternative options on supported hardware. Each successive algorithm exhibits an approximate 50% performance drop, with ChaCha performing the worst. Although AES-128 achieves higher throughput, AES-256 was selected for its stronger 256-bit key security.

2.1. Software architecture

Although WireGuard is optimized for in-kernel performance, it does not always scale efficiently in horizontally parallel (multi-core) deployments. This limitation primarily stems from kernel-level constraints such as spinlocks and soft interrupt (softIRQ) handling bottlenecks. As illustrated in Fig. 3, certain user-space implementations can outperform kernel-based WireGuard in multi-core environments, particularly when thread scheduling and memory locality are more effectively managed in user space.

2.2. Software functionalities

The Linux kernel natively supports AES algorithms, which are listed in `/proc/crypto`. We selected the `aes-aesni` algorithm by utilizing the `aesni_intel` module with a 256-bit block size. Although this selection is currently hardcoded in our implementation, runtime configuration is possible.

WireGuard, by default, implements its own ChaCha20-Poly1305 encryption. In our modified version, we replaced the ChaCha-based encryption and decryption routines with AES. Despite WireGuard's compact codebase, the AES encryption function is instantiated once using the kernel API `crypto_alloc_skcipher` within `device.c`. This function assigns the resulting cipher handle to the `multicore_worker` structure, which is later passed to the `encrypt` and `decrypt` functions.

Outgoing packets are processed in the `encrypt` function within `send.c`, where the original `chacha20poly1305_encrypt_sg_inplace` call is replaced with AES encryption logic. Specifically, a `skcipher_request` structure is allocated and populated with the encryption key and initialization vector (IV), followed by a call to `crypto_skcipher_encrypt`. The structure is subsequently destroyed to free resources.

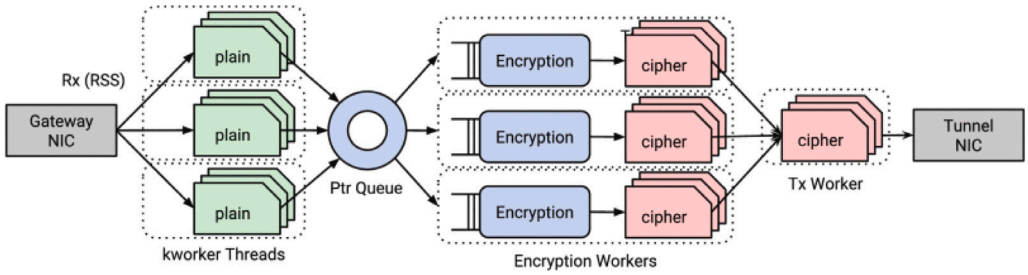

Fig. 3. WireGuard receive-side threading model in kernel-space and user-space configurations.

Table 3
TCP Traffic Generation Parameters and Metrics.

Parameter	Kernel (Plain)	Kernel (AES)
System	iPerf3	iPerf3
Protocol	TCP	TCP
Number of Runs	10	10
Packet Size (Bytes)	1310	1310
Packets Sent	2.9 Gp	3.3 Gp
Packets Received	2.9 Gp	3.3 Gp
Retransmission Rate	3.6%	3.4%
Simulation Time	1000 s	1000 s
Connections per Server	128	128
Number of Servers	25	25
Total Data Sent	3 TB	3.3 TB
Total Data Received	3 TB	3.3 TB

For incoming packets, decryption occurs in the `decrypt` function, which calls `crypto_skcipher_decrypt` using similar allocation and initialization logic.

2.3. AES-256-GCM integration in WireGuard

To support AES-256-GCM, we modified the WireGuard kernel module to enable the selection of the Authenticated Encryption with Associated Data (AEAD) cipher during both handshake negotiation and data packet processing. Specifically, we updated the following kernel code paths:

- `encrypt_packet()` in `device.c`, to support encryption using the `crypto_skcipher` interface;
- `decrypt_packet()` in `receive.c`, to verify AES-GCM authentication tags;
- `noise_handshake.c`, to allow handshake-derived keys to be used in AES-GCM contexts.

Encryption is implemented via the `crypto_skcipher` interface. A 256-bit key is retrieved from the session keypair using `keypair->sending.key`, and set through `crypto_skcipher_setkey()`. The initialization vector (IV) is constructed as a 12-byte array, with the last 8 bytes populated using the per-packet nonce: `PACKET_CB(skb)->nonce`. This format ensures IV uniqueness per packet, complying with GCM security requirements.

Listing 1 AES-256-GCM encryption logic in WireGuard

```
u8 *key = keypair->sending.key;
crypto_skcipher_setkey(tfm_aes_gcm, key, 32); // AES
    -256

u8 iv[12] = {0};
u64 nonce = PACKET_CB(skb)->nonce;
memcpy(iv + 4, &nonce, sizeof(nonce));

skcipher_request_set_crypt(request, sg, sg,
    plaintext_len, iv);
crypto_skcipher_encrypt(request);
```

The decryption path uses the symmetric function `crypto_skcipher_decrypt()` with `keypair->receiving.key` and the same IV construction method. This ensures that AES-GCM encryption and decryption are securely tied to session context and per-packet uniqueness.

All modifications are confined to the WireGuard kernel module and do not alter userspace behavior or interface semantics. The full implementation source code is publicly available at provided GitHub link.

2.4. AES-256-GCM integration in BoringTun

This study also utilized `BoringTun` as a user-space WireGuard implementation. `BoringTun` relies on the `ring` cryptographic library in the `Rust` programming language for encryption operations. This modular design enables straightforward replacement of the encryption algorithm during testing.

Specifically, within `session.rs`, the only required change is to import `AES_256_GCM` in place of `CHACHA20_POLY1305`. This substitution allows BoringTun to operate with AES-256-GCM instead of the default ChaCha20-Poly1305 cipher, enabling direct performance comparisons in user-space scenarios.

3. Illustrative examples

The network setup used for the experiments is shown in Fig. 4, and the traffic generation parameters are provided in Table 3. Although the system offered 88 logical CPU threads, we limited execution to 50 threads: 25 for the sender side and 25 for the receiver side.

The following WireGuard implementations were evaluated:

1. Kernel-based
   (a) Kernel-based WireGuard with software-based encryption
   (b) Kernel-based WireGuard with AES (+AES-NI support)
2. User-space-based
   (a) BoringTun with software-based encryption
   (b) BoringTun with AES (+AES-NI support)

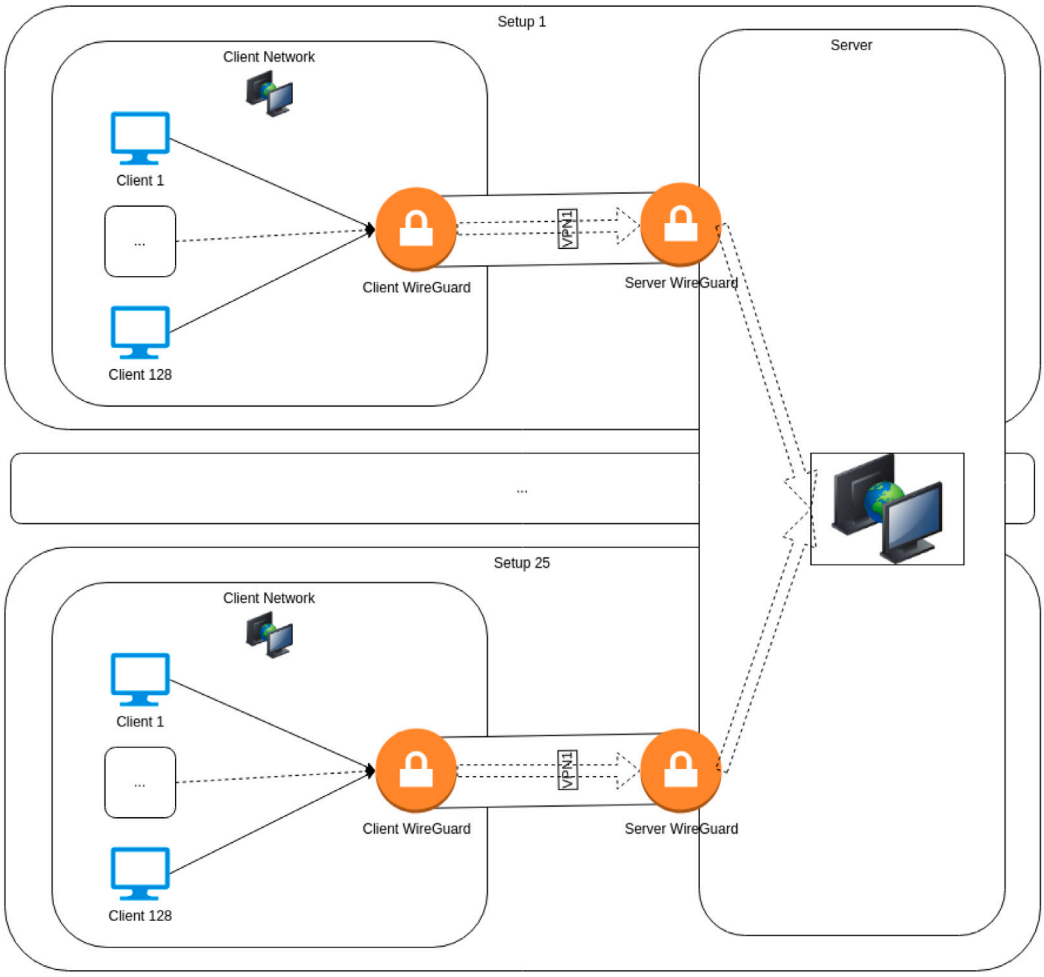

Fig. 4. Experimentation setup.

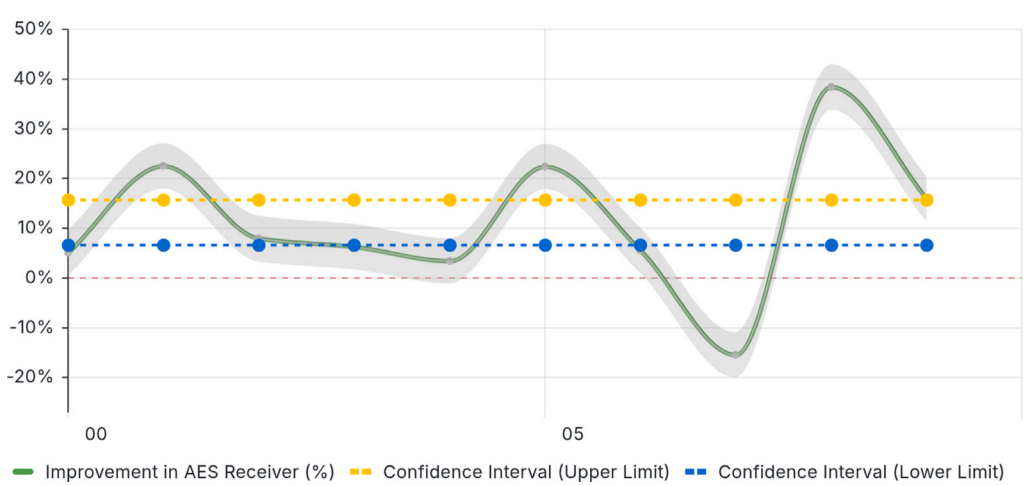

Fig. 5. 99% confidence interval for throughput (AES vs ChaCha20).

Table 4
Experimental Testbed Configuration.

Component	Specification
CPU	Intel 11th Gen Core i7-1165G7 @ 2.80 GHz (4 cores, 8 threads, Tiger Lake)
Memory	48 GB DDR4
NIC (Wired)	Realtek RTL8111/8168/8411 PCIe Gigabit Ethernet
NIC (Wireless)	Intel Wi-Fi 6 AX201 (`iwlwifi`, firmware: QuZ-a0-hr-b0-77.u)
Kernel	Linux 6.8.0-59-generic
Network tuning (sysctl)	`net.core.rmem_max` = 212992 `net.core.wmem_max` = 212992 `net.ipv4.tcp_rmem` = 4096 131072 6291456 `net.ipv4.tcp_wmem` = 4096 16384 4194304
IRQ handling	NIC-related interrupts (e.g., `iwlwifi` queues) distributed across all cores using the default scheduler; no manual CPU pinning applied

Table 5
BoringTun sender throughput (GB/s).

Trial	AES	ChaCha20	Difference (%)
1	2.5	2.3	9.2%
2	2.5	2.1	19.8%
3	2.5	2.0	23.6%
4	2.6	1.9	33.5%
5	2.4	2.1	16.5%
6	1.9	1.6	19.1%
7	1.9	1.7	15.5%
8	1.9	1.7	16.3%
9	2.0	1.6	22.1%
10	1.9	1.6	20.9%
Average	2.2	1.9	19.5%

Table 6
BoringTun receiver throughput (GB/s).

Trial	AES	ChaCha20	Difference (%)
1	2.5	2.3	9.2%
2	2.5	2.1	19.3%
3	2.5	2.0	23.7%
4	2.5	1.9	33.7%
5	2.4	2.1	16.0%
6	1.9	1.6	19.9%
7	1.9	1.7	15.5%
8	1.9	1.7	15.7%
9	2.0	1.6	22.8%
10	1.9	1.6	20.9%
Average	2.2	1.8	19.5%

Each implementation was tested with varying maximum transmission unit (MTU) sizes: 16, 256, 1350, 8192, and 16384 bytes. The resulting metrics for throughput, CPU usage, jitter (which was recorded as 17.9 s across all runs and is provided in the Grafana dashboard), and packet loss were recorded. However, delay measurements were not provided by `iperf3`.

3.1. Throughput

In the first trial, WireGuard was executed using the default kernel-based implementation without any source code modifications. In the second trial, we replaced the default ChaCha20 cipher with AES encryption. Each configuration was tested independently across ten repetitions to ensure statistical validity. As shown in Code metadata Table, all data and code used in these experiments are publicly available for external validation.

As previously noted, the peaks in Fig. 1 reflect client count variation. Initially, each of the 25 WireGuard server instances activates 128 clients in five-second intervals. Throughput stabilizes once all sessions are active, then shifts again near the 1000-second mark as clients disconnect.

Fig. A.2 present throughput comparisons across trials. AES achieved a throughput improvement of approximately 11% compared to ChaCha20. Average throughput was 20.2Gb/s for standard WireGuard and 22.3Gb/s for the AES variant, with a 99% confidence level (Fig. 5). Total retransmissions decreased by 4.5%, as shown in the Grafana dashboard provided in the code repository.

Tables 5 and 6 reflect measurements obtained using a high-end single-CPU system, while Table 7 presents results obtained in the same distributed environment used for kernel-based testing. The influence of CPU power is evident (AES performance improves with better hardware). All BoringTun benchmarks were performed on a consumer-grade laptop running Ubuntu 22.04 with the specifications defined in Table 4. Power-saving features and CPU frequency scaling were disabled via the `performance` governor to maintain consistent benchmarking conditions.

3.2. CPU usage

Processors with hardware-accelerated encryption support demonstrate improved performance by offloading cryptographic operations from software to hardware. This not only reduces overall encryption latency but also frees up CPU cycles for other user-space or kernel-space tasks.

Table 8 presents the average CPU usage during BoringTun trials using both ChaCha20 and AES backends. Notably, while the AES-enabled configuration performs better in throughput, it also shows

Table 7
BoringTun per-client throughput (Mb/s).

Client Trial	Sent (ChaCha)	Recv. (ChaCha)	Sent (AES)	Recv. (AES)
1	779	779	801	800
2	758	758	818	818
3	761	760	845	844
4	766	766	888	888
5	759	759	820	819
6	758	758	868	867
7	762	762	828	827
8	779	779	817	817
9	759	759	805	804
10	760	760	856	856
11	768	768	830	829
12	752	752	816	816
13	766	766	854	854
14	759	759	810	809
15	767	767	826	825
16	756	755	806	806
17	769	769	846	846
18	755	755	818	817
19	786	786	807	807
20	756	756	865	865
Avg.	763.8	763.7	831.2	830.7
Diff.	–	–	+67.1	+67.5
Percentage	–	–	8.8%	8.9%

Table 8
Per-sample CPU usage comparison (%) using BoringTun over 50 samples. Mean and 95% confidence intervals (CI) are provided to support the reported CPU usage differences.

Sample	ChaCha	AES	Sample	ChaCha	AES
1	43.8	75	26	81.2	100
2	37.5	75	27	106.7	100
3	100.0	93.8	28	100.0	93.3
4	93.3	87.5	29	93.8	93.8
5	100.0	100.0	30	93.8	87.5
6	93.8	87.5	31	100.0	93.3
7	93.8	106.7	32	80.0	86.7
8	93.8	93.3	33	106.7	100.0
9	93.3	93.8	34	93.3	93.8
10	93.3	87.5	35	93.8	93.8
11	87.5	100.0	36	81.2	87.5
12	81.2	100.0	37	100.0	100.0
13	100.0	93.8	38	93.8	86.7
14	93.3	87.5	39	93.3	93.8
15	100.0	100.0	40	80.0	87.5
16	87.5	87.5	41	106.2	100.0
17	100.0	100.0	42	75.0	73.3
18	86.7	100.0	43	106.7	100.0
19	93.8	100.0	44	93.3	87.5
20	81.2	93.8	45	100.0	106.7
21	100.0	106.7	46	87.5	93.3
22	87.5	100.0	47	100.0	87.5
23	93.3	100.0	48	81.2	87.5
24	93.3	87.5	49	93.8	106.7
25	87.5	100.0	50	95.6	90.3
Mean	94.0	91.8	95% CI	±2.9	±2.2
CPU usage: ChaCha20 averaged 2.2% higher CPU with a wider CI range.					

slightly reduced average CPU consumption (91.8% vs 94.1%) indicating an approximately 2.2% improvement. This reflects more efficient utilization of CPU resources due to hardware offloading (e.g., via AES-NI), despite occasional higher peaks depending on thread scheduling and system load.

3.3. Impact of higher MTUs

As shown in Fig. 2, AES performance improves with increasing block sizes. Since the Maximum Transmission Unit (MTU) governs packet size at the network layer, higher MTUs reduce cryptographic overhead and enhance throughput.

In our experiments, AES-enabled configurations performed significantly better with MTUs of 8192 and 16384 bytes than with the default 1350 bytes. Larger payloads amortize encryption and system call overhead, improving per-byte efficiency. AES-NI further benefits from batching on larger blocks.

In controlled environments without fragmentation risks (e.g., data centers or point-to-point tunnels), larger MTUs offer a practical optimization. Future work will provide detailed throughput and CPU metrics across MTU and cipher configurations.

4. Impact

WireGuard has seen widespread adoption due to its high performance, security-by-design architecture, and ease of configuration. Although it outperforms many legacy VPN systems (including those with hardware-accelerated encryption) it still faces architectural limitations in high-throughput or large-scale deployments. Enhancing VPN throughput is especially critical for real-time systems, where latency and jitter are consequential.

Experimental results show that increasing WireGuard client connections on a single server does not yield linear throughput gains. This nonlinearity stems from the kernel-based architecture's limited parallelization, constrained by spinlocks, shared caches, and softIRQ overhead.

Client-side performance is generally adequate using the default kernel implementation. However, server-side VPN gateways (VPNGWs) aggregating hundreds or thousands of connections benefit significantly from AES-NI-accelerated encryption. While software-based AES can degrade performance on less capable clients, the impact is typically tolerable; AES-NI systems, however, achieve substantial efficiency improvements under high load.

Fig. 1 indicate that AES outperforms ChaCha20 by 15%, even at low connection counts. Kernel-mode WireGuard exhibits performance dips during session ramp-up and teardown phases, whereas user-space implementations such as BoringTun maintain stable 1Gb/s throughput per client on single-core systems (highlighting a resilience not seen in the kernel variant).

Flame graphs (e.g., Fig. A.1) show that encryption and decryption account for <10% of CPU usage. Most overhead originates from the kernel's networking stack, conntrack, and netfilter layers. Offloading encryption to dedicated hardware reveals further optimization opportunities.

In summary, accelerating WireGuard's cryptographic layer via AES-GCM and AES-NI yields measurable gains in throughput and CPU utilization, especially in high-connection server scenarios. These findings underscore the importance of kernel-level optimization for scalable, high-speed VPN infrastructures.

5. Conclusions

Modern mobile networks increasingly adopt IP-based architectures to enhance core security. Within this context, WireGuard stands out for its simplicity, compact codebase, and strong cryptographic foundations, making it suitable for integration into mobile and vehicular systems. Its relevance extends to underexplored Vehicle-to-Everything (V2X) scenarios.

Kernel-based WireGuard, combined with AES-NI, can outperform ChaCha20 in high-performance *EDGE* server environments, improving both throughput and CPU efficiency. However, broader adoption in carrier-grade settings depends on achieving terabit-scale throughput from a single instance (currently unfeasible). This study shows that such scalability is more attainable with AES and hardware accelerators like FPGAs or application-specific integrated circuits (ASICs).

A core limitation of WireGuard is the absence of cipher negotiation. Our AES-GCM implementation requires both client and server to use the same cipher. A more flexible protocol could dynamically select between AES-GCM and ChaCha20-Poly1305 based on client capability, though this would impose extra overhead on the server. A hybrid model may be necessary to balance compatibility and performance.

Future work will explore user-space packet processing via eBPF/XDP and VPP/DPDK, which can bypass kernel bottlenecks and enhance control over memory locality and thread scheduling. Eliminating kernel constraints such as softIRQ handling and spinlocks may further improve WireGuard's scalability in multi-core systems.

Our next research phase targets VPP/DPDK-based WireGuard implementations where encryption efficiency becomes pivotal. The AES integration developed in this study offers a strong foundation for these optimizations. Preliminary per-CPU and per-worker enhancements under development show promise and will be presented in future work.

CRediT authorship contribution statement

M.F. Yuce: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **O. Keskin:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **E. Yerlikaya:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **M.Y. Akmaz:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Investigation. **A. Kirca:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **D. Yiltas-Kaplan:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **M.A. Erturk:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Z. Gurkas-Aydin:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Ö.C. Turna:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Ş. Durukan-Odabaşı:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **M.A. Aydin:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the EUREKA CELTIC-NEXT IMMINENCE project (Ref. No: 2021-02107, Grant No: SEK 10,589,723). Additional support was provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK) under the 2219 International Postdoctoral Research Fellowship Program (Application No: 1059B192301494).

We also acknowledge the funding and resources provided by Istanbul University–Cerrahpaşa Scientific Research Projects Commission, Türkiye under Project No: 36754. Our gratitude extends to the Istanbul University–Cerrahpaşa Scientific Research Projects Unit for their continuous support.

This research was further supported by Ulak Haberleşme A.Ş. as part of the Next Generation SDN Project. Finally, this study constitutes a portion of the Ph.D. dissertation titled *Development of a Hybrid Intrusion Detection System Model Proposal in Autonomous Vehicle Networks* at Istanbul University, Institute of Physical Sciences.

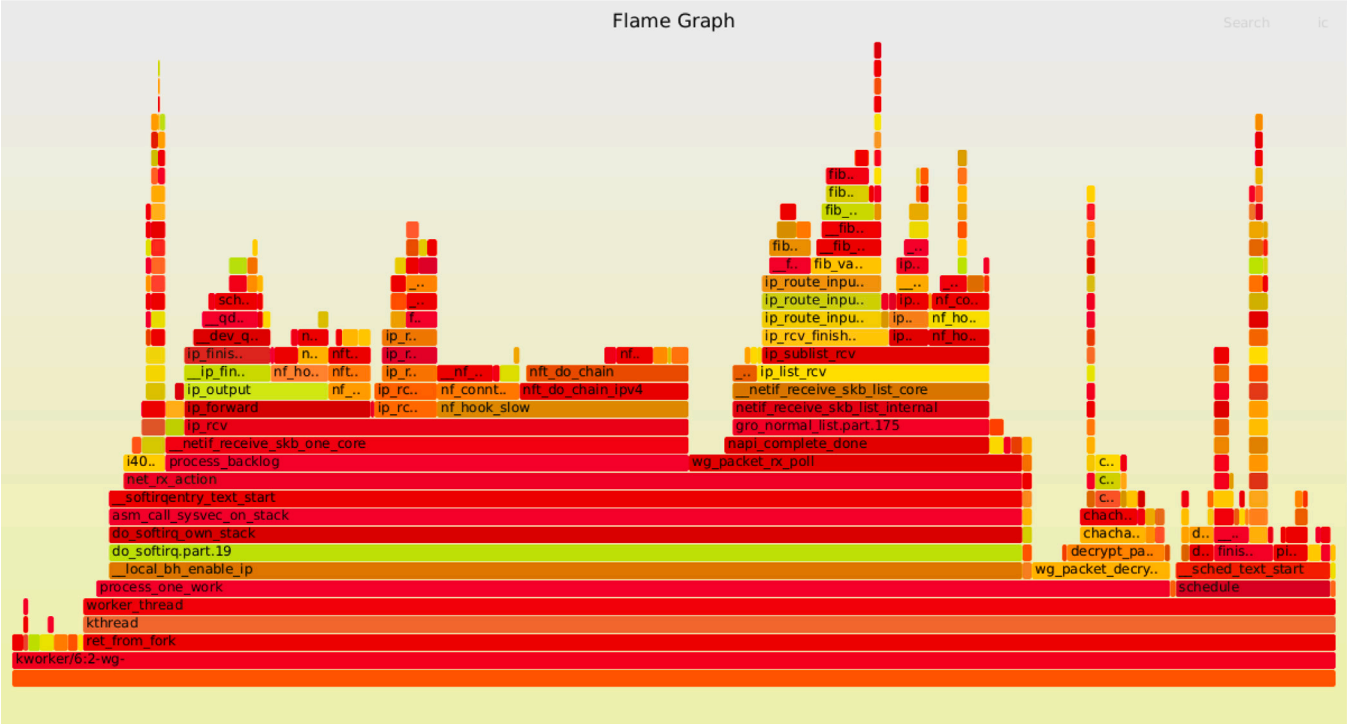

Fig. A.1. Flame graph of the Linux networking stack with WireGuard on the receiver side.

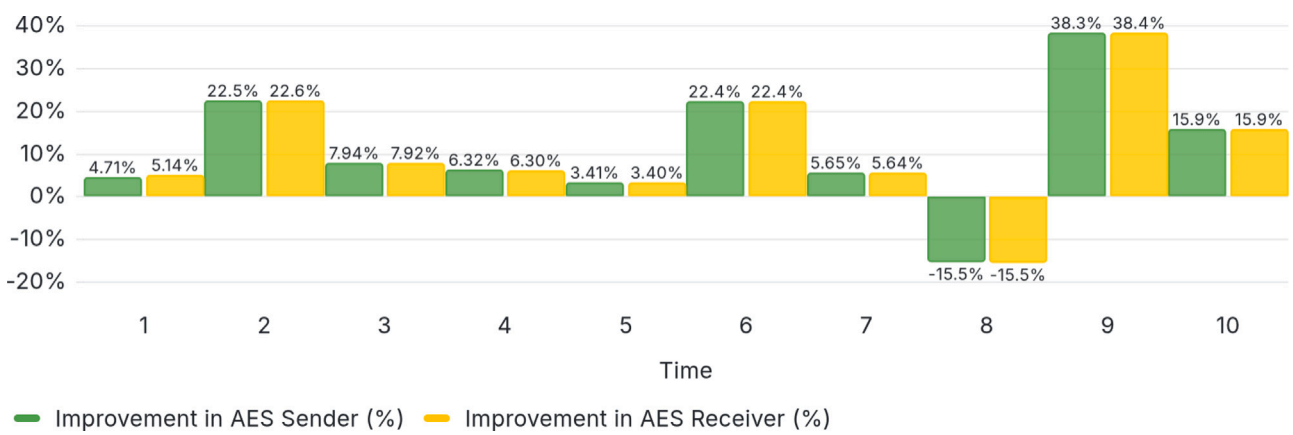

Fig. A.2. Throughput by run for AES and ChaCha20 variants.

Appendix. Additional visuals

See Figs. A.1 and A.2.

References

[1] Donenfeld JA. WireGuard: Next generation kernel network tunnel. In: The network and distributed system security (NDSS) symposium. 2017, p. 1–12.

[2] Perrin T. The noise protocol framework. 2018, URL http://www.noiseprotocol.org/noise.pdf.

[3] Dowling B, Paterson KG. A cryptographic analysis of the WireGuard protocol. In: International conference on applied cryptography and network security. Springer; 2018, p. 3–21.

[4] Donenfeld JA, Milner K. Formal verification of the wireguard protocol. Technical report, 2017, Available online URL https://www.wireguard.com/papers/wireguard-formal-verification.pdf.

[5] Lipp B, Blanchet B, Bhargavan K. A mechanised cryptographic proof of the WireGuard virtual private network protocol. In: 2019 IEEE European symposium on security and privacy. IEEE; 2019, p. 231–46.

[6] Abdulazeez A, Salim B, Zeebaree D, Doghramachi D. Comparison of VPN protocols at network layer focusing on WireGuard protocol. Int J Interact Mob Technol (IJIM) 2020;14(18).

[7] Mackey S, Mihov I, Nosenko A, Vega F, Cheng Y. A performance comparison of WireGuard and OpenVPN. In: Proceedings of the tenth ACM conference on data and application security and privacy. 2020, p. 162–4.

[8] Leffler SJ. Fast IPSec: A high-performance ipsec implementation. In: BSD conference and expo. 2003, p. 133–40.

[9] Vajaranta M, Oinonen A, Hämäläinen TD, Viitamäki V, Markunmäki J, Kulmala A. Feasibility of FPGA accelerated IPsec on cloud. Microprocess Microsyst 2019;71:102861.

[10] Park MH, Beom MJ, Park WK, Jeong YK, Paik EH. Implementation and performance evaluation of hardware accelerated ipsec VPN for the home gateway. In: The 7th international conference on advanced communication technology, 2005, vol. 2, IEEE; 2005, p. 1007–10.

[11] Brunella MS, Belocchi G, Bonola M, Pontarelli S, Siracusano G, Bianchi G, et al. hXDP: Efficient software packet processing on FPGA NICs. Commun ACM 2022;65(8):92–100.

[12] Zhang RF, Tkachuk G, De Lara Guarch P, Kantecki T, Ji K, DiGiglio J. FD.io VPP-SSwan and linux-CP - integrate strongswan with world's first open sourced 1.89 Tb IPsec solution. Tech. rep., Intel Network Builders; 2023, URL https://builders.intel.com/docs/networkbuilders/fd-io-vpp-sswan-and-linux-cp-integrate-strongswan-with-world-s-first-open-sourced-1-89-tb-ipsec-solution-technology-guide-1686047475.pdf.

[13] Shantharama P, Thyagaturu AS, Reisslein M. Hardware-accelerated platforms and infrastructures for network functions: A survey of enabling technologies and research studies. IEEE Access 2020;8:132021–85.

[14] Pudelko M, Emmerich P, Gallenmüller S, Carle G. Performance analysis of VPN gateways. In: 2020 IFIP networking conference. IEEE; 2020, p. 325–33.

[15] Liu J, Gao N, Tu C, Zhang Y, Sun Y. A pure hardware design and implementation on FPGA of WireGuard-based VPN gateway. In: 2023 26th international conference on computer supported cooperative work in design. IEEE; 2023, p. 1220–5.

[16] Kane LE, Chen JJ, Thomas R, Liu V, Mckague M. Security and performance in IoT: A balancing act. IEEE Access 2020;8:121969–86.

[17] Dekker E, Spaans P. Performance comparison of VPN implementations WireGuard, strongswan, and OpenVPN in a 1 gbit/s environment. 2020, URL https://www.os3.nl/_media/2019-2020/courses/rp2/p71_report.pdf.